



Full-Stack Software Development

Lab 9: Custom Hooks & I18n

R. Naudts, J. Pieck, J. Rombouts, B. Van Impe

Contents

1 INTRODUCTION	3
2 GETTING STARTED	4
3 ADDING A CUSTOM HOOK	5
4 ADD INTERNATIONALIZATION	6
4.1 MIDDLEWARE	6
4.2 TRANSLATION FILES	6
4.3 CONFIG LOADER	6
4.4 NEXTINTLCLIENTPROVIDER	7
4.5 USE AND GETTRANSLATIONS.....	7
4.6 CREATE A WAY TO SWITCH LANGUAGES.....	7

1 Introduction

In this lab, we will first build our own custom hook.

In the second part we will add Spanish translations to our application



The concepts covered include:

- ✓ Reusing stateful logic with custom hooks
- ✓ Internationalization



For an in-depth explanation of the concepts we will be applying in this lab, please consult these two presentations:

- 01 - Custom Hooks
- 03 – Internationalization
- From previous lectures: 01 – react context

2 Getting started

To begin this lab, please accept the following GitHub assignment:

<https://classroom.github.com/a/AgeGus4w>

Once you have done this, you will be provided with a repository containing the starter code. Clone it to a local directory and open it in VSCode.

Start the back-end application

This lab requires the backend-lecturers application you implemented in **lab 7 & 8** to be running.

Prepare the front-end application

1. Open the cloned project in VSCode.
2. Install all required dependencies by running the following command in the terminal:

```
npm install
```

Connect the front-end to the back-end

Our front-end application needs to know the URL of our back-end API. We will provide this using an environment variable.

1. In the root of your frontend-lecturers project, create a new file named: `.env`
2. Add the following content to this file:

```
NEXT_PUBLIC_API_URL=http://localhost:3000
```

3. Start your Next.js development server:

```
npm run dev
```

4. Open `http://localhost:8080` in your browser

3 Adding a custom hook

In the previous lab, we added a react context and every time we needed the user, login or login method, we need the same lines of code:

```
const context = useContext(AuthContext);
if (!context)
  throw new Error('UserLoginForm must be used within an AuthProvider');
const { user, logout, login } = context;
```

Instead of copy-pasting these same lines of code, we should extract them out into a custom hook.

1. From the AuthContext file, move the AuthContextType to the types/index.tsx file. Make sure to export it. Add the import in to the AuthContext.

```
import { User, AuthContextType } from '@types';
```

2. Create a new folder called 'hooks' in the root of the application and add a file called useAuth.tsx
3. Using the slides and the code above: write a custom hook that encapsulates the logic above.
4. Use your custom hook everywhere the useContext is used now:

```
const { user, Logout, Login } = useAuth();
```

5. Verify that everything works as expected.



- ✓ Refactor the Authcontext file
- ✓ Create a custom hook
- ✓ Call the custom hook in the code
- ✓ **Optional:** could you implement this functionality in another way than with a custom hook? If so, how?

4 Add Internationalization

Following the steps from the slides, we'll add i18n to a part of our application. Before we can start, we need to install **next-intl**.

```
npm i next-intl
```

4.1 Middleware

In the root of the project, add a `middleware.ts` file (not `tsx`!) and add the code for the locales "en" and "es". Use the same matcher as in the slides. This middleware will read the URL and determine the locale of the request.

Make sure to add a dynamic route to all pages in your app folder.

4.2 Translation Files

These are already provided in the code. Take a look at the `common.json` files in `public > locales`.

4.3 Config loader

Add a file with the name `i18n.ts` to the root of the project. This file loads the `common.js` file corresponding with the current locale.

Create the `next.config.js` file so it contains the `withNextIntl` method

4.4 NextIntlClientProvider

Open the root layout.tsx file and add the NextIntlClientProvider. This provider is what will provide the translations to our components.

4.5 Use and getTranslations

- In server components, replace hardcoded UI text strings with translations, using `getTranslations()`
- In client components, replace hardcoded UI text strings with translations, using the `useTranslation` hook.

Do this at least for the Header, the UserLoginForm and the homepage.

4.6 Create a way to switch languages

The Translation component is already present. You just need to implement the `handleLanguageChange` function and add the component to the header.

Now try it out in your browser.



- ✓ Set up all necessary config to use translations
- ✓ Use actual translations in components
- ✓ Make a language switcher component
- ✓ **Optional:** adding another language is less work. Try to work out what you need to do.